Objects and editing

A flow subtracts from one named account and adds to another. The application uses Python dictionaries and one SQLite repository. These are the actual objects returned by Repository.get_account() and get_flow().

```
{
"name": "AmEx",
"opening_cents": 0,
"opening_at": "2026-10-01 00:00:00"
}

{
"flow_id": "11111111-1111-4111-8111-111111111111",
"date": "2026-10-02 12:30:00",
"from_account": "AmEx", "to_account": "Joe's Diner",
"amount_cents": 1200, "status": "Complete",
"name": "Friday lunch", "notes": "Burger and fries",
"category": "Dining", "tags": ["Road trip"]
}
```

Representation	Meaning
Account name	Its identity. Starting balance is integer cents, negative for debt, or null when unknown. New endpoint accounts start unknown.
Amount / flow ID	1200 means $12.00; exact integer arithmetic. UUID preserves identity during edits/imports. Repeat creates a new UUID.
Date	2026-10-02 or 2026-10-02 12:30:00. Time is optional; entered time normalizes to seconds. A date alone means midnight for calculations. No separate flag.
Category / tags	One category and a list of tag names. Shared tags can connect split purchases. Nine flow fields plus ID; no scenario field.

The GUI edits these objects

Date, From, To, Amount, Status, Name, Notes, Category, Tags. Defaults: today, blank time, Complete and Uncategorized. Accounts has Name, Starting Balance and Starting Date, with optional time.

Notes: inline and expanded editors share one draft. Close, Escape and window close retain it; Up/Down follows wrapped lines and the wheel scrolls. Save commits notes with the entire flow; Cancel discards the draft.

Click a ledger row to load it. Save updates that ID and clears the editor; Cancel clears without saving. Leaving a draft asks: Save changes to this flow? Yes / No / Cancel. Record JSON shows the selected saved record.

Repository-generated objects and rows: examples/record_objects.json and examples/review.sql. Launching and controls: README.md. Source walkthrough: docs/model_review.md. Forecast collections and simulation controls are deferred.

Storage and calculations

Named keys are name.strip().casefold(): AmEx becomes amex. They provide case-insensitive matching and foreign-key references, not separate generated identities. Display spelling stays in name.

Table	Every stored column
accounts	account_key, name, opening_cents, opening_at
flows	flow_id, date, from_key, to_key, amount_cents, status, name, notes, category_key
categories	category_key, name
tags	tag_key, name
flow_tags	flow_id, tag_key, position
audit	audit_id, changed_at, entity, entity_key, action, before_json, after_json

Category/tag tables retain choices; flow_tags preserves assignments and order. audit stores saved before/after values; audit_id is its sequence. Data > View Schema reads the live definitions. Constraints and indexes: flowledger/schema.sql.

Save and calculate

GUI draft -> Repository.save_flow() -> validate -> write choices, flow, tags and audit in one transaction -> return the saved dictionary. Failed saves roll back together. core.py owns storage and math; exchange.py imports/exports; gui.py and widgets.py manage drafts and views.

An account receives +amount when To and -amount when From. Confirmed includes Complete; Including Pending adds Pending; Including Planned also adds Planned. Void contributes zero. Settle Pending by editing that record; a refund is a new reverse flow.

Each account has one confirmed starting balance immediately before flows at its timestamp. Earlier Complete flows are already reflected; eligible earlier Pending/Planned flows enter estimates once. Unknown balances remain unknown. Dates are points in local time, compared at second precision; date-only means midnight.

Graphs compare or sum explicitly selected accounts. A transfer cancels within a sum containing both endpoints. Selecting every endpoint conserves the total; this is not automatically net worth. Category totals count each matching non-Void flow once, including transfers.

Upgrade and scope

Schema 1-3 upgrades first save a full .pre-v4-timestamp.bak. Old Baseline flows stay active with their IDs; accounts and history remain. Old hypothetical collections remain in the backup and .bak.forecasts.json recovery file, outside the active ledger. Time flags become date precision, then disappear.

Import/export: nine fields plus ID. Old scenario/time-flag imports are rejected; upgrade the database and export again. Use Backup for a complete snapshot. Scope: USD cents, one starting balance per account, local timestamps, one computer. Executed checks: docs/test_results.txt and docs/interaction_check.txt.